

Fuzzy EWS

Grid Search & AUROC Evaluation Technical Documentation

*How the temporal context builder parameters are optimised
and how predictive performance is evaluated*

Table of Contents

1. Overview
2. The Three Parameters
3. Grid Search Script (grid_search_auroc.py)
 - 3.1 Pipeline Architecture
 - 3.2 Stage 1: Data Loading & Cleaning
 - 3.3 Stage 2: Pre-compute Fuzzy Scores
 - 3.4 Stage 3: Pre-compute OLS Trend Slopes
 - 3.5 Stage 4: The Grid Search Loop
 - 3.6 Stage 5: Scoring Formula
 - 3.7 Output & Visualisations
4. AUROC Evaluation Script (auroc_optimal.py)
 - 4.1 What It Does Differently
 - 4.2 Point-Estimate AUROC
 - 4.3 Patient-Level Bootstrap CIs
 - 4.4 Sensitivity at Fixed Specificity
 - 4.5 Output Plots
5. Key Formulas Reference
6. Appendix: Grid Search Visualisations

1. Overview

The Fuzzy Early Warning System (EWS) uses fuzzy logic to convert raw vital-sign readings into concern scores, then applies a temporal context builder to incorporate trend information. Two scripts handle optimisation and evaluation:

- **grid_search_auroc.py:** Searches over a 3D parameter grid to find the combination of alpha, beta, and gamma that maximises AUROC for predicting REVIEW_WITHIN_4HOURS.
- **auroc_optimal.py:** Takes the winning parameters and runs a deeper evaluation with bootstrap confidence intervals, sensitivity/specificity analysis, and publication-quality plots.

Both scripts compare three predictors against the same binary label (REVIEW_WITHIN_4HOURS = 1 if the patient needed clinical review within 4 hours):

- NEWS-2 (pre-computed integer score from the dataset)
- Snapshot Fuzzy EWS (sum of 6 per-vital fuzzy scores, no temporal adjustment)
- Temporal Context Builder (EWMA + trend sigmoid + aggregation blend)

2. The Three Parameters

Alpha (EWMA smoothing factor)

Controls how much weight the latest observation receives versus historical observations. At $\alpha=1.0$ only the current observation matters (no smoothing). At $\alpha=0.1$ the score is heavily smoothed, carrying forward historical values.

$$\text{EWMA}[t] = \alpha * \text{raw}[t] + (1 - \alpha) * \text{EWMA}[t-1]$$

Grid: 0.1, 0.2, 0.3, ..., 1.0 (10 values, step 0.1)

Beta (sigmoid trend steepness)

Controls how aggressively a worsening trend inflates the score. Higher beta means even small positive slopes produce a large trend adjustment. The trend factor uses a modified sigmoid that is zero when the slope is non-positive.

$$\text{trend_factor} = 2 / (1 + \exp(-\beta * \text{slope})) - 1 \quad [\text{only when slope} > 0]$$

Grid: 0.5, 1.0, 1.5, ..., 5.0 (10 values, step 0.5)

Gamma (aggregation blend)

Controls how the six per-vital adjusted scores are combined into a single total. At $\gamma=1.0$ it is a simple additive sum (range 0-18). At $\gamma=0.0$ it would be purely max-based (the single worst vital scaled to 0-18). Values in between blend the two strategies.

$$\text{total} = (1 - \gamma) * (18/3) * \text{max_vital} + \gamma * \text{sum}(\text{all_vitals})$$

Grid: 0.1, 0.2, 0.3, ..., 1.0 (10 values, step 0.1)

Total search space: $10 \times 10 \times 10 = 1,000$ parameter combinations.

3. Grid Search Script (grid_search_auroc.py)

3.1 Pipeline Architecture

The script is designed for efficiency on a ~9.3 million row dataset. The key optimisation principle is: pre-compute everything that does not depend on the three search parameters (alpha, beta, gamma), so the inner loop only performs fast vectorised arithmetic.

The pipeline has five stages:

Stage 1: Load & clean data	[one-time]
Stage 2: Pre-compute per-vital fuzzy scores	[one-time, parameter-independent]
Stage 3: Pre-compute OLS trend slopes	[one-time, parameter-independent]
Stage 4: Grid search loop	[1,000 iterations]
Stage 5: Score each combination with AUROC	[inside loop]

3.2 Stage 1: Data Loading & Cleaning

The script loads the training CSV and applies several cleaning steps:

- Coerces vital-sign columns to numeric (handles 'Refused', blanks, etc.)
- Filters to rows where COMPLETE_DATA = 1
- Drops rows with NaN vitals after coercion
- Casts columns to efficient types (float32, int8) to reduce memory
- Clamps inspired O2 to [21, 100]
- Constructs t_minutes (continuous timeline) from DAYS_SINCE_ADMISSION + OBS_TIME
- Sorts by patient ID then time, so per-patient operations work correctly

3.3 Stage 2: Pre-compute Fuzzy Scores

For each of the six vital signs, the script builds a lookup table (LUT) that maps every possible input value to a defuzzified concern score (range 0 to 3). The defuzzification process is:

- Load the sigmoid membership functions from CSV files in the generated_membership_data/sigmoid/ directory.
- For each input value in the membership function grid, evaluate all membership degrees (e.g., 'Below normal - severe concern' through 'Above normal - severe concern').
- Map the input memberships to four output concern levels: No concern, Mild, Moderate, Severe (using max-aggregation within each level).
- Defuzzify using centroid method over output membership functions defined as trapezoids on the range [0, 3].

Once the LUT is built, all 9.3M observations are scored via numpy interpolation (np.interp), which is nearly instant. This avoids running the full fuzzification pipeline 9.3M times.

The six vitals and their membership function types are:

Vital	Column	MF Type
Heart rate	HEART_RATE	7-variable
Blood pressure	SYSTOLIC_BP	7-variable
Temperature	TEMPERATURE	7-variable
Respiratory rate	RESP_RATE	7-variable
Oxygen saturation	SATS_SPO2	3-var (downward)
Inspired oxygen	INSPIRED_O2_TEXT	3-var (upward)

3.4 Stage 3: Pre-compute OLS Trend Slopes

For each vital sign and each observation, the script computes the ordinary least squares (OLS) slope of the fuzzy score within a 24-hour look-back window. This captures whether the vital is trending better or worse.

The computation works as follows:

- For each patient, maintain a sliding window of observations within the past 24 hours (1440 minutes).
- Convert times to hours relative to the window start, then compute the OLS slope: $\text{beta} = \frac{\sum((t - t_mean)(s - s_mean))}{\sum((t - t_mean)^2)}$.
- Require at least 2 observations in the window to compute a slope.
- A positive slope means the concern score is increasing (patient worsening).

This is the most expensive pre-computation step (iterates per-patient, per-observation), but it only runs once because slopes do not depend on alpha, beta, or gamma.

3.5 Stage 4: The Grid Search Loop

The three parameters are searched in a nested loop, ordered from most expensive to cheapest recomputation:

```
for alpha in [0.1, 0.2, ..., 1.0]:      <-- outermost (10 iters)
  compute EWMA for all 6 vitals          <-- expensive: per-patient sequential
  clamp: clamped = max(EWMA, raw_score)

  for beta in [0.5, 1.0, ..., 5.0]:      <-- middle (10 iters)
    compute trend-adjusted scores        <-- moderate: vectorised sigmoid

    for gamma in [0.1, 0.2, ..., 1.0]:   <-- innermost (10 iters)
      aggregate to single total           <-- cheap: weighted sum
      compute AUROC                      <-- sklearn roc_auc_score
```

Why this nesting order matters:

- **Alpha outermost:** The EWMA depends only on alpha and requires a sequential pass over each patient's timeline. By placing alpha outermost, we compute the EWMA only 10 times total (once per alpha value), not 1,000 times.
- **Beta middle:** The sigmoid trend adjustment depends on alpha (via the clamped EWMA) and beta. It is vectorised element-wise arithmetic, moderately cheap.
- **Gamma innermost:** The gamma aggregation is just a weighted sum of arrays that are already computed. This is the cheapest operation, so it runs the most iterations (innermost).

3.6 Stage 5: Scoring Formula

For a given (alpha, beta, gamma), the per-vital adjusted score is computed as:

Step A: EWMA Smoothing

```
ewma[t] = alpha * raw[t] + (1 - alpha) * ewma[t-1]
```

The EWMA is then clamped so it can only raise the score, never lower it:

```
clamped[t] = max(ewma[t], raw[t])
```

Step B: Trend Adjustment

A sigmoid-based trend factor is applied only when the OLS slope is positive (worsening trend). The factor ranges from 0 (no trend) toward 1 (strong trend).

```
trend_factor = 2 / (1 + exp(-beta * slope)) - 1    [slope > 0 only]
```

The adjusted score combines the clamped EWMA with the trend factor:

```
adjusted = clamped + trend_factor * (3.0 - clamped)
```

The term (3.0 - clamped) means the trend pushes the score toward the maximum of 3. Scores already near 3 have less room to increase. The result is clipped to [0, 3].

Step C: Aggregation Across Vitals

The six per-vital adjusted scores are combined using gamma:

```
additive      = sum of all 6 adjusted scores      (range 0-18)
```

```
max_based     = (18/3) * max(adjusted scores)     (range 0-18)
```

```
total         = (1-gamma) * max_based + gamma * additive
```

At gamma=1.0 the total is a simple sum. At lower gamma values, the single worst vital dominates the total score.

Step D: Evaluate with AUROC

Each combination's total score array is evaluated with sklearn's `roc_auc_score` against the binary label `REVIEW_WITHIN_4HOURS`. The combination achieving the highest AUROC is selected as optimal.

3.7 Output & Visualisations

The grid search produces the following outputs in `grid_search_results/`:

- **grid_search_results.csv**: CSV file with all 1,000 (alpha, beta, gamma, AUROC) combinations.
- **heatmaps.png**: Three 2D heatmaps showing AUROC for each pair of parameters (alpha-beta, alpha-gamma, beta-gamma), with the third fixed at its optimal value.
- **surfaces_3d.png**: Same pairwise views as 3D surface plots with the optimal point marked in red.
- **sensitivity.png**: 1D plots varying each parameter while fixing the other two at optimal.
- **top_n_configs.png**: Horizontal bar chart of the top 20 parameter configurations by AUROC.
- **parameter_distributions.png**: Mean, min, and max AUROC marginalised over each parameter.
- **roc_comparison.png**: Full ROC curves comparing NEWS-2, Snapshot Fuzzy, and the best Temporal model.

4. AUROC Evaluation Script (auroc_optimal.py)

4.1 What It Does Differently

While the grid search evaluates 1,000 configurations quickly, `auroc_optimal.py` takes the single best configuration and runs a thorough statistical evaluation. It imports core functions from `grid_search_auroc.py` to avoid code duplication.

The optimal parameters (found by grid search) are hardcoded at the top of the file:

```
ALPHA = 0.70
BETA  = 0.5
GAMMA = 0.80
```

4.2 Point-Estimate AUROC

The script computes all three predictor arrays (NEWS-2, Snapshot Fuzzy, Temporal Builder) and calls sklearn's `roc_auc_score` for each. This gives a single AUROC number per predictor.

How `roc_auc_score` works internally:

- Rank all observations by predicted score, from highest to lowest.
- Sweep a threshold from the maximum score down to the minimum. At each threshold, compute TPR (sensitivity) and FPR (1 - specificity).
- This traces out the ROC curve (TPR vs FPR).
- Compute the area under that curve using the trapezoidal rule.

Interpretation: AUROC = 0.669 means that if you randomly pick one positive observation and one negative observation, there is a 66.9% chance the model assigns the positive a higher score. An AUROC of 0.5 = random guessing; 1.0 = perfect separation.

Importantly, AUROC does not choose a threshold. It is purely about ranking: do positive cases tend to get higher scores than negative cases?

4.3 Patient-Level Bootstrap Confidence Intervals

A single AUROC number does not tell you how certain you can be. The script constructs 95% confidence intervals using patient-level bootstrapping (200 resamples by default).

Why patient-level, not observation-level?

- Observations from the same patient are correlated (if one reading is abnormal, adjacent readings likely are too). Treating them as independent would underestimate the variance of the AUROC estimate.
- Patient-level resampling keeps all of a patient's observations together, correctly accounting for within-patient correlation.

The bootstrap procedure:

- Identify all unique patient IDs and pre-index their observation rows.
- For each of 200 iterations: sample N patients with replacement (where N = total unique patients), gather all their observations, compute AUROC on this resample.
- Take the 2.5th and 97.5th percentiles of the 200 AUROC values as the 95% CI.

4.4 Sensitivity at Fixed Specificity

This answers the clinical question: 'If I want to correctly rule out X% of healthy patients (specificity), what proportion of deteriorating patients will I catch (sensitivity)?'

The script evaluates at four specificity targets: 80%, 85%, 90%, 95%.

- Compute the full ROC curve (arrays of FPR, TPR, and thresholds).
- Convert FPR to specificity: $\text{specificity} = 1 - \text{FPR}$.
- For each target specificity, find the closest point on the curve and report the corresponding sensitivity and score threshold.

4.5 Output Plots

The evaluation script produces four plots in `auroc_results/`:

- **roc_comparison.png**: Full ROC curves for all three predictors with AUROC and 95% CI in the legend.
- **roc_zoomed.png**: ROC zoomed to the high-specificity region (FPR 0 to 0.3), where clinical decisions typically operate.
- **precision_recall.png**: Precision-Recall curves, which are more informative than ROC under severe class imbalance (0.79% positive rate).
- **score_distributions.png**: Histograms of score distributions for positive vs negative observations, showing how well the scores separate the two classes.

A summary CSV (`auroc_summary.csv`) is also saved with:

Model	AUROC	CI Lower	CI Upper	Avg Precision
NEWS-2	0.6570	...	...	...
Snapshot Fuzzy	0.6660	...	...	...
Temporal Builder	0.6693	...	...	...

5. Key Formulas Reference

Trapezoid Membership Function

$$\mu(x; a, b, c, d) = \{ (x-a)/(b-a) \text{ if } a < x < b; 1 \text{ if } b \leq x \leq c; (d-x)/(d-c) \text{ if } c < x < d; 0 \text{ otherwise} \}$$

Centroid Defuzzification

$$\text{output} = \text{integral}(x * \mu_{\text{agg}}(x)) / \text{integral}(\mu_{\text{agg}}(x))$$

Where μ_{agg} is the aggregated output membership function (Mamdani-style: clip each output MF at its firing strength, then take the pointwise max).

EWMA (Exponentially Weighted Moving Average)

$$\text{ewma}[t] = \alpha * \text{raw}[t] + (1 - \alpha) * \text{ewma}[t-1]$$

OLS Trend Slope (within 24h window)

$$\text{slope} = \text{sum}((t_i - t_{\text{mean}})(s_i - s_{\text{mean}})) / \text{sum}((t_i - t_{\text{mean}})^2)$$

Sigmoid Trend Factor

$$\text{trend_factor} = 2 / (1 + \exp(-\beta * \text{slope})) - 1 \quad [\text{slope} > 0]$$

Per-Vital Adjusted Score

$$\text{adjusted} = \max(\text{ewma}, \text{raw}) + \text{trend_factor} * (3.0 - \max(\text{ewma}, \text{raw}))$$

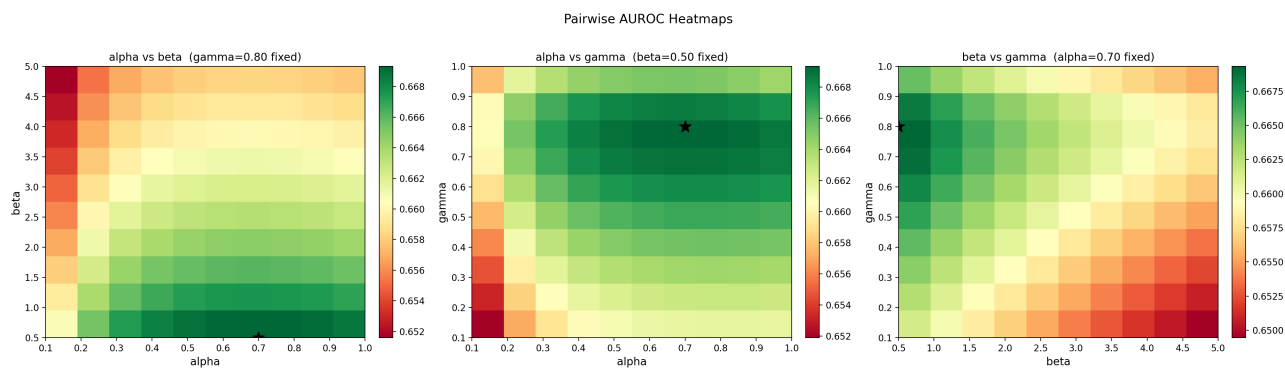
Total Score Aggregation

$$\text{total} = (1 - \gamma) * (18/3) * \text{max_vital} + \gamma * \text{sum}(\text{adjusted})$$

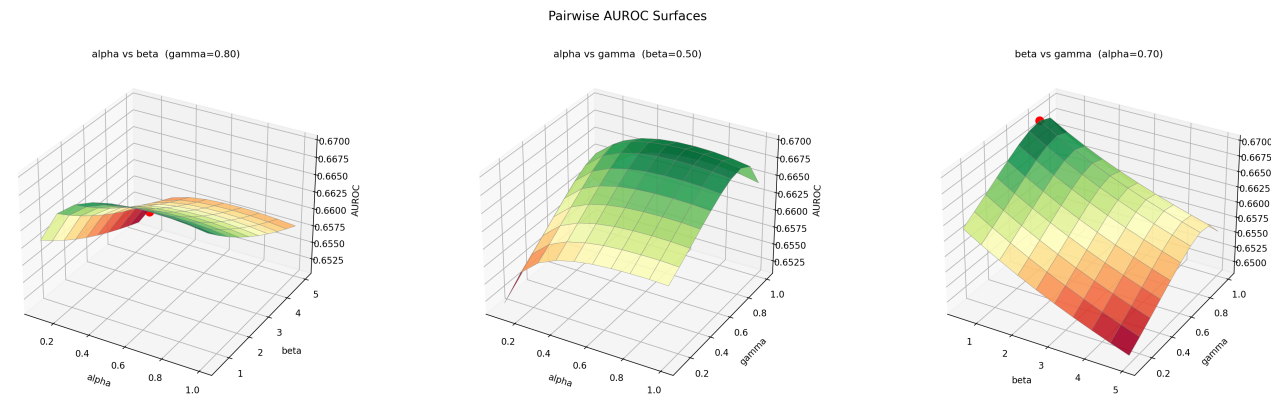
AUROC (Area Under ROC Curve)

$P(\text{score}(\text{random positive}) > \text{score}(\text{random negative}))$. Computed by sweeping all possible thresholds and integrating TPR over FPR.

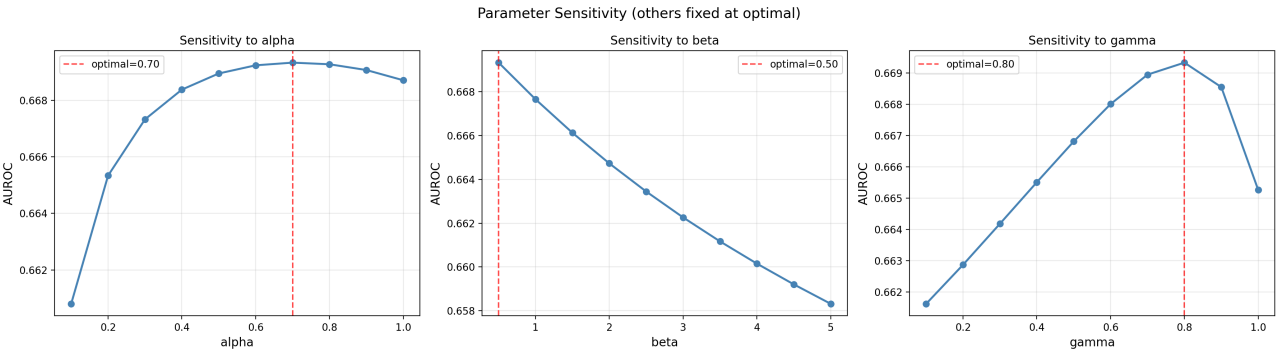
Appendix: Pairwise AUROC Heatmaps



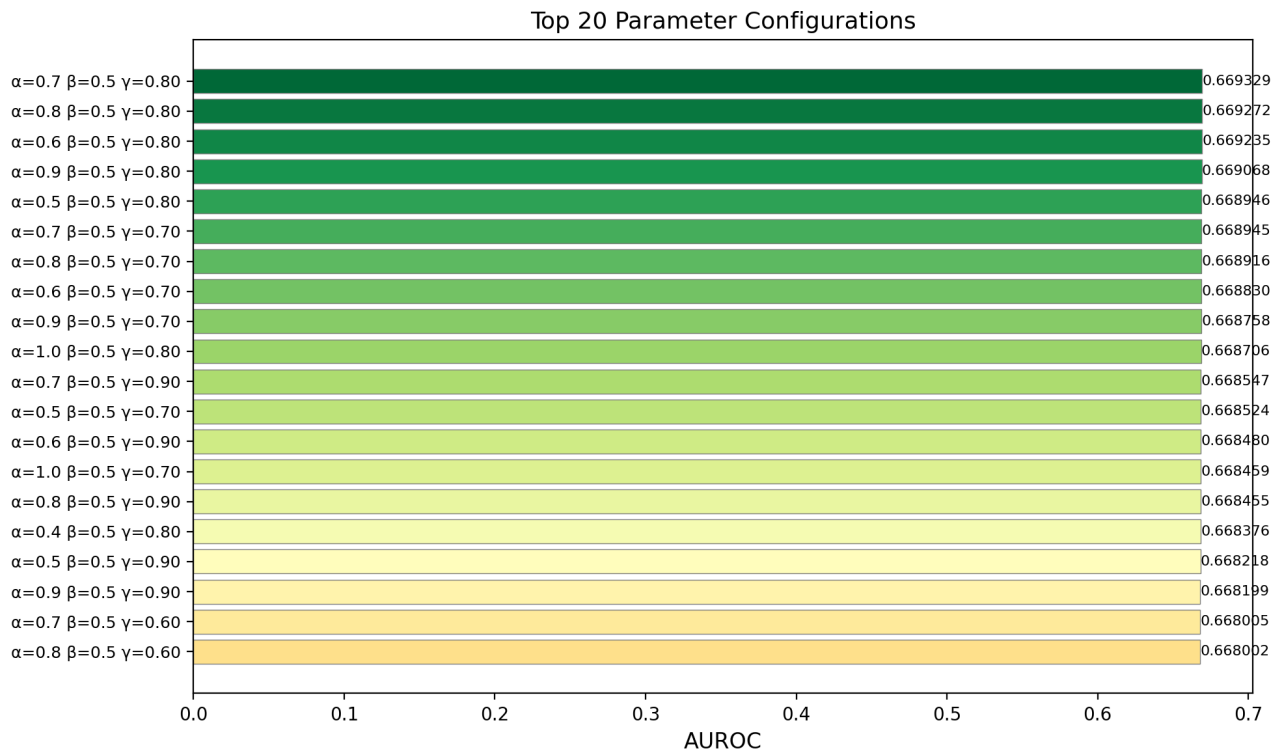
Appendix: Pairwise AUROC 3D Surfaces



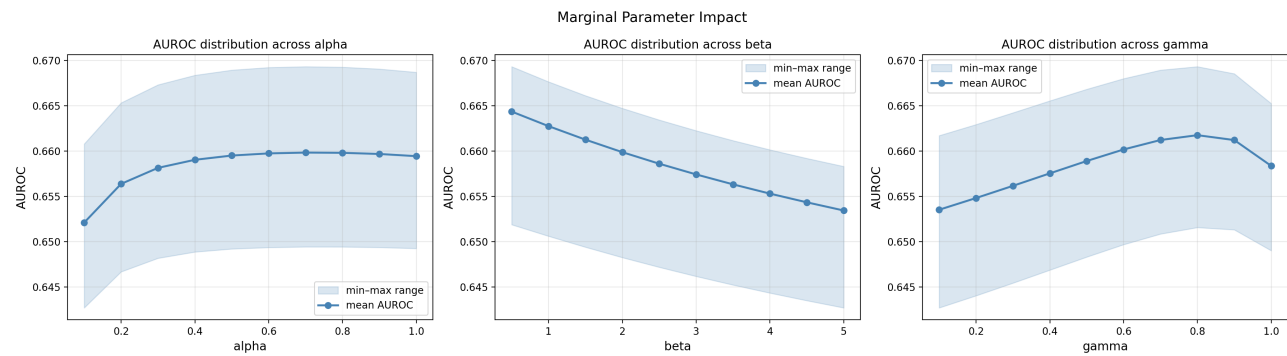
Appendix: Parameter Sensitivity Plots



Appendix: Top 20 Parameter Configurations



Appendix: Marginal Parameter Impact



Appendix: ROC Curve Comparison

